keys, and the values in the codomain are indices from `0` to `M-1`.

Since the target set of a hashing function is typically smaller than the source set, there will be collisions: at least 2 values will be mapped to the same index. Hash tables, as we have seen in chapter 2, use a few strategies to resolve conflicts, like *chaining* or *open addressing*.

The other important thing to keep in mind is that we distinguish hash maps and hash sets: the formers allow us to associate a value[5] to a key; the latter, instead, only record the presence or absence of a key in a set. Hash sets implements a special case of *dictionary*, the `Set`. With respect to our definition of *dictionary* as an abstract data structure, given at the beginning of this section, a *set* is a specialization of a dictionary, where the `value` type is set to `Boolean`: the second parameter to `insert` becomes redundant, as the value associated with a key in the hash set will be implicitly assumed to be `true`.

Abstract Data Structure: Set	
API	`class Set { insert(key) remove(key) contains(key) → true/ false }`
Contract with client	A set maintains *a set of keys*. If a key K was added to the set (and not removed afterwards), then `contains(K)` will return `true`, otherwise it will return `false`.

As we explain in appendix C, all operations in a hash table (and hash set) can be performed in amortized `O(1)` time.